De La Salle University-Manila
College of Computer Studies
2401 Taft Avenue Malate, Manila

Term X, A.Y. 202X - 202X

AISHA: AI Support for Hires and Associates

Technical Write-Up

In partial fulfillment of the course requirements for

<COURSE> <SECTION> - <FULL COURSE CODE>

Submitted by:

<Member 1>

<GROUP MEMBERS>

Submitted to:

<Prof_Name>, PhD

July 09, 2026

Table of Contents

This contents page is intentionally simple so it remains easy to edit in Word. Page numbers are filled from the rendered draft where available.

Section	Focus	Page
1. Business Case	Why AISHA matters for Alyssa's time-to-ramp	3
2. Methodology	How the local-first prototype was built	4
3. Architecture	Inputs, process, outputs, tools, safety, and monitoring	6
4. Experiments and Evaluation	What was tested, what passed, what failed, and what we learned	10
5. Retrospective	Human reflection on what worked and what we would change	12
References	Sources cited in the write-up	13
Appendix A. Implementation Evidence Map	Supporting implementation and test evidence	14

1. Business Case

To begin with, AISHA means AI Support for Hires and Associates, and this project treats onboarding as a lived transition rather than a stack of forms. Importantly, BDO AISHA is simply an educational capstone prototype for a fictionalized BDO onboarding story, and it is not an official BDO system. Hence, in this write-up, all employee records, handbook content, org contacts, metrics, and demo conversations are invented for learning, testing, and demonstration purposes.

To tell the story, the idea began with a feeling that is easy to recognize if you have ever entered a large company for the first time. Earlier, one of us experienced that environment through a Shopee internship, and later, both of us got into P&G. At first, the exciting part is obvious: the company name feels big, the opportunity feels real, and you want to prove that you belong. However, after that excitement settles, the quieter part appears, and it is not because one is incapable. Instead, it is because one can get lost when the map of responsibilities is split across people, portals, group chats, policy pages, acronyms, and expectations.

That is the gap AISHA tries to answer. Specifically, in large organizations, onboarding time is not lost only because new hires cannot find a document. More often, time is lost because the new hire does not know which document matters, which person owns the blocker, or whether a question is "too basic" to ask. In the literature, onboarding research shows that early work tasks shape not only output, but also confidence, learning, and socialization (Ju et al., 2021). Likewise, recent work on graduates points to mentorship as a major need for students moving into professional life (Whalley et al., 2024). And that is why we believe AISHA is beneficial as it is built around that messy middle: the space between being hired and feeling useful.

In the demo, that new hire is Alyssa Reyes, a Management Trainee and Branch Banking Associate. Her first meaningful goal is not to finish a generic checklist or survive a long onboarding folder. Instead, her goal is to reach a Day 30 Readiness Check, where she should be ready for supervised branch customer interactions with process awareness, compliance awareness, and enough confidence to know who to ask when something is unclear. This makes the business case sharper because AISHA is not trying to be a general HR chatbot. Rather, it is trying to reduce time-to-ramp for someone who has to learn policies, systems, branch routines, workplace norms, and human relationships at the same time.

In practice, AISHA's value comes from connecting four pieces that are usually separated. First, it gives grounded answers from the fictional handbook and cites the source. Second, it reads and updates the new hire's actual ramp plan. Third, it routes questions to named people, such as IT access, payroll, compliance learning, the manager, or the buddy. Finally, it gives HR support signals without exposing raw private chat transcripts by default. Individually, these features are useful. Together, they turn onboarding from a scavenger hunt into a guided route.

However, this privacy boundary is not a small detail. AI in HR can quickly feel uncomfortable when it becomes a way to watch people rather than help them. For example, studies on attitudes toward non-human resource management found concerns around AI management, emotional monitoring, and workplace surveillance (Mantello et al., 2021). Similarly, research on AI-driven HRM shows that the field is expanding across analytics, talent management, and workforce planning, which makes governance and trust part of the design problem (Maghsoudi et al., 2023). Because of that, AISHA chooses a support-not-surveillance frame. HR should see delayed milestones, unresolved blockers, concern tags, pulse trends, and suggested actions, but not Alyssa's full private chat by default.

One helpful analogy is a check-engine light. The light does not shame the driver, publish the whole trip, or guess someone's character. Instead, it simply says that something needs attention before a small issue becomes expensive. AISHA's pulse check-ins work the same way. When a weekly sentiment score is low or declining, the purpose is not to label Alyssa as a risk. The purpose is to suggest a buddy check-in, clarify expectations, unblock access, or file a People Experience escalation before the situation turns into quiet disengagement.

Technically, this is why a plain chatbot was not enough. A chatbot can answer a question, but AISHA needs to do several connected things: retrieve handbook passages, cite them, remember Alyssa's ramp state, update tasks, route people, log support signals, and apply guardrails. Retrieval-augmented generation, or RAG, fits this need because it lets a language model answer with help from retrieved documents instead of answering from memory alone (Lewis et al., 2020). In addition, RAG surveys describe retrieval as a practical way to reduce unsupported or outdated model responses (Gao et al., 2023). Put simply, AISHA is designed to take an open-book exam, not guess from memory.

Ultimately, the business case is not "AI for HR because AI is trendy." It is a quieter argument: onboarding is a coordination problem. Alyssa needs the handbook, the plan, the right human owner, and a safe way to say when she is stuck. At the same time, HR needs enough signal to offer help, but not enough detail to police her. AISHA sits between those two needs. It acts less like a judge with a scorecard and more like a patient guide with a map, a clipboard, and a phone directory.

2. Methodology

To understand the methodology, it helps to admit the constraint first. We were students building a working agentic AI system, not an enterprise implementation team with a budget, cloud contracts, HRIS access, and months of runway. Because of that, our method was not to choose the most impressive stack possible. Rather, it was to keep asking one practical question: what is the smallest honest system that can prove AISHA's support loop?

From there, we started with Alyssa's journey before choosing the technology. What would she need in her first month? She would need clear answers, a role-based ramp plan, named owners, task updates, a way to escalate, and a weekly check-in that does not feel like a performance review. Human-centered AI work argues that AI systems should support human goals, control, and trust rather than automate blindly (Shneiderman, 2020). That became our north star. AISHA should help Alyssa move through onboarding, not replace the people responsible for helping her.

The first major stack decision was local-first AI. Admittedly, a cloud model could have been faster and more powerful. If this were a funded pilot, we would seriously consider Azure OpenAI, OpenAI, Anthropic, or another enterprise provider because they offer stronger hosted infrastructure, monitoring, and access controls. For this capstone, though, cloud AI added cost, internet dependence, and privacy awkwardness. Ollama let us run the model locally, which made the demo easier to rehearse offline and easier to explain: the fictional chat, retrieval, database, and logs stay on the machine unless we deliberately connect them elsewhere.

Next, we chose RAG over fine-tuning. Fine-tuning sounds attractive because it feels like teaching the model to become AISHA. In practice, it was too much work for the value we needed. It would cost more, take longer to iterate, and still would not solve the citation problem cleanly. RAG was simpler and more honest. AISHA retrieves from the fictional handbook, then answers with a citation

such as `[source: leave_policy.md]`. That matters because a new hire should not have to trust a confident answer without knowing where it came from.

For retrieval storage, we used Chroma. Think of it as a small local filing cabinet for handbook chunks. We could have used Pinecone, Weaviate, Elasticsearch, Azure AI Search, or another production-grade search layer. Those would make more sense for a real company with thousands of documents, permissions, audit trails, and document owners. For a capstone, Chroma was enough because our handbook is intentionally small, local, and easy to rebuild after edits. This kept the retrieval layer understandable instead of turning the project into an infrastructure exercise.

For employee state, we used SQLite. This was a practical choice, not a glamorous one. AISHA needed to remember employees, ramp tasks, completed milestones, escalations, pulse check-ins, and persisted chat messages. In the long run, PostgreSQL would be the better database, especially with migrations and access control. For this demo, though, SQLite worked like a durable notebook inside the project folder. It gave us continuity across restarts without making setup painful.

Meanwhile, LangChain and LangGraph handled the agent loop. We chose them because AISHA needed to decide which tool to use: search the handbook, read the plan, complete a task, find a person, or escalate to HR. We did not want to hard-code every possible conversation path. At the same time, we did not let the model control everything. When a user asks to complete a task, deterministic code checks for ambiguity before updating SQLite. That division matters because the language model is useful for conversation, but ordinary code is safer for state changes, matching, logging, and citation enforcement.

In addition, guardrails were part of the build from the start. The input guardrail checks whether a message is on-topic, off-topic, or a prompt-injection attempt. The output guardrail checks citations and redacts obvious number-shaped private information. These are not perfect safety measures, and we should not pretend they are. Still, the NIST AI Risk Management Framework treats AI as a socio-technical system, which means risk depends on the model, the users, the setting, and the lifecycle around it (National Institute of Standards and Technology, 2023). AISHA follows that idea in a small way: define the scope, cite the handbook, avoid repeating sensitive numbers, and keep human escalation available.

For the interface, we chose Streamlit because we needed a working app quickly. A custom React or Next.js interface would look better and scale better, but it would also take time away from the actual agentic behavior. Streamlit let us show both sides of the loop in one place: Alyssa's chat and the HR support dashboard. The tradeoff is that the current UI still feels like a demo. A future version should turn the new-hire view into a Day 30 readiness cockpit and make the HR view more card-based, focused on support actions rather than tables.

Similarly, FastAPI was added so AISHA would not be trapped inside one interface. The API proves that the same guarded pipeline can be reused outside Streamlit. That matters because real onboarding systems usually live across portals, learning platforms, communication tools, dashboards, and identity systems. In this project, the API is a small bridge toward that future rather than just a requirement checkbox.

Observability was another practical layer. We used local JSONL logs instead of a heavier tool like MLflow, LangSmith, or OpenTelemetry. The log records route, model names, latency, estimated token counts, tools used, retrieved sources, guardrail category, plan changes, escalation IDs, and

errors. Importantly, it does not log raw message text. That choice keeps the system inspectable without turning monitoring into a transcript archive.

Finally, Docker rounded out the methodology because a project that only runs on one student's laptop is fragile. Docker packages the app environment so the setup is less mysterious for another person. We did not bundle Ollama inside the image because local models are large and hardware-dependent. Instead, the container points to a host Ollama server. It is not the fanciest deployment story, but it is honest for the scope.

Several smaller choices also mattered. The simulated date is threaded through the app so pulse check-ins can be demoed without waiting weeks. Tests use fake LLMs so the suite can run without Ollama. The citation format is treated as a contract across retrieval, prompting, and guardrails. A rebrand test prevents old pre-AISHA wording from coming back. These details are not flashy, but they make the project easier to trust.

In the end, the course module checklist became a scaffold rather than the product itself. Prompt engineering, structured outputs, RAG, memory, guardrails, ReAct, tool use, chat UI, API, observability, and Docker all appear in AISHA. Each one has a job in the story. Structured outputs make pulse and guardrail results predictable. Memory lets Alyssa's progress survive the next turn. Tool use lets the assistant act instead of merely answer. The methodology was to build a small complete loop, then keep the parts visible enough that we could explain, test, and improve them.

With more time or budget, we would upgrade the stack in predictable ways: enterprise identity, PostgreSQL, managed retrieval, consent and retention controls, stronger monitoring, HRIS and LMS integrations, calendar signals, and real role-based access. For this capstone, we chose the stack that made the learning visible. AISHA is local, grounded, stateful, tool-using, guarded, observable, and packaged. More importantly, it proves the central idea: a new hire does not need a magic oracle. She needs the next useful step, the right person, and a safe way to say, "I am stuck."

3. Architecture

To understand AISHA's architecture, it helps to start with the project promise instead of the tools. The system is built around one person, Alyssa Reyes, and one practical milestone: her Day 30 Readiness Check for supervised branch customer interactions. Everything in the architecture exists to help her move toward that point with less confusion, more confidence, and clearer support from the people around her.

At a high level, AISHA is a local-first agentic AI system. "Local-first" means the main demo components run on the machine: the Ollama language models, the Chroma vector store, the SQLite database, the Streamlit interface, and the local JSONL logs. "Agentic" means AISHA does more than generate a paragraph. It can retrieve policy context, inspect Alyssa's ramp plan, mark tasks complete, route her to the right person, file an escalation, remember chat history, and feed privacy-preserving support signals into the HR view.

The first diagram shows the whole system as input, process, and output. It is intentionally broad. Instead of reading it as a wiring diagram, read it as the story of what AISHA receives, what it prepares, and what it gives back.

Figure 1. Input-process-output system architecture

INPUT	PROCESS	OUTPUT
Static Knowledge - docs .md - policy docs	1. Knowledge Preparation - ingest - chunk - embed - Chroma vector store	User-Facing Output - grounded response - citations - ramp plan - task updates
Seeded Data - employee JSON - plan JSON - org JSON	2. Context Assembly - retrieve knowledge - load user state - load memory/history	HR/Admin Output - escalations - pulse trends - HR dashboard
Persistent State - chat memory - pulse history - escalations	3. Agent Reasoning + Tools - guardrail classifier - LangGraph/ReAct agent - local LLM - agent tools	System Side Effects - updated memory - updated SQLite - JSONL logs
Runtime Input - user message - demo clock	4. Safety + Monitoring - output guardrails - PII redaction - observability	Supported Loop - new-hire support - HR signal - local audit trail

Note: Figure is built as an editable Word table so labels and boxes can be adjusted without redrawing the diagram.

To begin with, the input side is split into four kinds of material. Static knowledge is the fictionalized BDO onboarding handbook stored as Markdown files. This includes policies, first-day guides, payslip explainers, IT setup notes, benefits summaries, branch ramp guidance, and glossary material. Seeded data gives the demo its cast and structure: employee records, role-based ramp plans, and a fictional org directory. Persistent state is what changes over time, such as chat memory, completed tasks, pulse check-ins, and escalation tickets. Runtime input is the live turn itself, plus the simulated date that drives the demo clock.

That simulated date is a small detail with a large effect. In a real company, support signals appear over days or weeks. In a classroom demo, we cannot wait a week to show a pulse check-in. So the Streamlit sidebar lets us move the demo clock forward, and the system threads that date into the agent prompt and pulse scheduling logic. This keeps the story honest. We are not pretending time passed in production. We are deliberately simulating time so the support loop can be demonstrated.

Once the inputs exist, AISHA prepares them in two different ways. The handbook goes through the knowledge preparation path. The ingestion script loads Markdown files, reads simple front matter, splits the text into chunks, embeds those chunks with the local Ollama embedding model, and stores them in Chroma. In plain language, this turns the handbook into searchable memory. When Alyssa asks, "What does AML mean?" or "How do I set up MFA?", AISHA does not have to rely on the language model's general memory. It can search the local handbook and answer from retrieved text.

Meanwhile, employee and ramp data follow the state path. SQLite stores Alyssa's profile, plan items, completed tasks, escalations, chat messages, and pulse history. We chose direct repository methods instead of letting the model write SQL because state changes should be boring and controlled. The language model can decide that a task completion tool is useful, but ordinary code decides which task matches and whether the request is too ambiguous to update. That division protects the demo from a common agent risk: a confident model taking the wrong action.

In practice, the center of the architecture is context assembly. Every turn gathers the things AISHA needs to answer as Alyssa's assistant, not as a generic chatbot. The system prompt includes her name, role, department, start date, manager, buddy, simulated date, and current week of ramp. Recent chat history is added so the conversation can continue naturally. The tools are rebuilt for the current employee and date, which means a call to `get_my_plan` or `complete_task` is always scoped to the signed-in demo persona.

After context assembly, the agent decides what to do. AISHA uses a LangChain/LangGraph ReAct-style agent with ChatOllama. ReAct means the model can reason about the request, choose a tool, observe the result, and then compose an answer. We used this pattern because Alyssa's questions are not all the same shape. Sometimes she needs a policy answer. Sometimes she needs to know what is next in her plan. Sometimes she is blocked by access and needs a human owner. Sometimes the safest answer is to escalate rather than guess.

The tools are deliberately limited to five. `search_knowledge_base` retrieves handbook chunks and captures source metadata. `get_my_plan` reads Alyssa's ramp plan. `complete_task` updates a plan item, but only after deterministic matching and ambiguity checks. `find_person` looks up fictional BDO contacts for things like IT access, payroll, benefits, compliance learning, branch shadowing, manager touchpoints, or buddy support. `escalate_to_hr` creates a People Experience ticket when the handbook has no answer or the user needs human help.

That tool set is where the business value becomes visible. A normal chatbot might say, "You should ask IT." AISHA can search the IT setup guide, cite it, find the named IT support owner in the org directory, and, if the situation is unresolved, file an escalation. A normal checklist app might show Alyssa a list of tasks. AISHA can explain why the task matters for Day 30 readiness, mark the right item complete, and refresh the progress view. This is why the architecture is not just technical decoration. The pieces are arranged around time-to-ramp.

However, tool use also creates responsibility. If a user says "mark branch practice done," there may be multiple similar tasks in the plan. AISHA should not silently choose the wrong one. The `complete_task` tool uses deterministic scoring over task titles, and if two open tasks are too close, it returns an ambiguous result instead of mutating state. Then the assistant asks which task ID or title the user meant. This is a small feature, but it expresses the design philosophy: when the system can act, it should also know when to pause.

Safety wraps both sides of the agent. Before a normal chat message reaches the agent, the input guardrail classifies it as on-topic, off-topic, or prompt injection. If the user asks for a university essay or tries to override the system prompt, AISHA refuses politely and steers back to onboarding support. The guardrail is intentionally fail-open if the classifier breaks, because a broken local model should not make the whole demo unusable. That is a demo tradeoff. In production, we would likely use stronger policy layers and more conservative failure behavior.

After the agent writes an answer, output guardrails run as well. If AISHA used the knowledge base, the answer must include `[source: filename.md]` citations. If the model forgets, the guardrail appends the retrieved sources. If the system searched and found no relevant document, the answer should say the handbook does not cover the question and offer escalation. The output pass also redacts obvious number-shaped private information, such as long account-like or ID-like numbers, if the assistant tries to repeat them back.

At the same time, AISHA records observability metadata in a JSONL file. The log includes route, model names, latency, estimated token counts, tools used, retrieved sources, guardrail category, escalation ID, plan-change status, and errors. It deliberately does not log raw message text. That matters for the privacy thesis. Monitoring should help us debug the system and evaluate the demo, but it should not become a hidden transcript archive.

The second diagram zooms into one AI turn. It shows the path from Alyssa's message to the rendered answer, including pulse replies, routing, tools, output guardrails, persistence, and logging.

Figure 2. AI turn flow

Step	What AISHA Does	Main Routes or Outputs
1. Receive Turn	User message or API request is attached to the employee record and simulated date.	Raw turn saved to SQLite chat memory.
2. Pre-Route Checks	AISHA checks whether the message is a pulse reply or a normal chat turn.	Pulse replies go to scoring. Normal turns go through input guardrails.
3. Intent Router	The agent infers what Alyssa is trying to do and chooses tools when needed.	Knowledge -> search KB Plan -> get plan Task -> complete task People -> find person Sensitive gap -> escalate to HR
4. Response + Safety	The agent composes an answer, then output guardrails enforce citations and redact number-shaped PII.	Grounded answer, refusal, or escalation response.
5. Persist + Render	Streamlit or FastAPI returns the answer and records metadata.	Updated memory, plan state, sources panel, escalation toast, JSONL log.

Note: The figure is intentionally simplified. The detailed behavior lives in the prose and source code, while this table keeps the turn flow editable.

The turn begins in either Streamlit or FastAPI. Streamlit is the main demo surface because it lets us show Alyssa's chat and the HR dashboard in one place. FastAPI proves that the same guarded pipeline can be reused outside the UI, which matters if AISHA were later connected to another portal, workflow, or internal tool. In both cases, the system identifies the employee, attaches the simulated date, loads recent memory, and prepares the turn.

The first special branch is the pulse check-in. If AISHA has just asked a weekly well-being question, the next reply is treated as a pulse response instead of a normal policy question. This is important because a pulse reply might be emotional, vague, or broad. It should not be rejected as off-topic just because it sounds unlike a handbook question. Instead, the pulse classifier converts the reply into a sentiment score, concern tags, and a short privacy-preserving summary for HR.

Here again, the architecture follows the support-not-surveillance rule. HR can see that Alyssa may be blocked on tools, unclear about expectations, or declining in sentiment. HR does not need to see every private sentence she typed by default. The dashboard is meant to create a humane opening for help, not a record for judgment. In the current prototype, raw pulse replies are still stored locally for demo drill-down, which we treat as a known gap. A production version would need consent, retention rules, and role-based access before exposing that detail.

For normal chat messages, the input guardrail runs before the agent. If the message is allowed, the agent receives recent history and decides which tool path fits. A knowledge question triggers retrieval. A progress question triggers the plan tool. A completion request triggers task matching and possible mutation. A routing question triggers the people directory. A sensitive issue or handbook gap can trigger escalation. The user does not have to know those routes. Alyssa can simply ask in normal language, and the agent chooses the path.

The response then passes through the safety layer and returns to the surface. In Streamlit, the answer is rendered in the chat with an expandable Sources panel when documents were retrieved. If a task changed, the progress display refreshes. If an escalation was filed, the app shows a toast and the HR dashboard can see it. In the API, the response returns structured fields: answer, citations, sources, escalation ID, plan-change status, guardrail category, and refusal status. The same underlying loop supports both.

Ultimately, the architecture produces three kinds of output. Alyssa receives grounded answers, citations, plan guidance, task updates, people routing, and a safer way to say she is stuck. HR receives support signals: open escalations, delayed progress, pulse trends, concern tags, and short summaries. The system itself records side effects: updated SQLite state, persisted chat memory, refreshed pulse history, and privacy-conscious JSONL run logs.

What this architecture does not do is just as important. It does not connect to real BDO systems. It does not use real employee data. It does not authenticate with production SSO. It does not read attendance systems, calendars, LMS platforms, payroll systems, or HRIS records. Those would be future integrations, and they would raise serious governance questions. For the student prototype, the honest scope is a local fictional demo that proves the support loop without pretending to be enterprise software.

We could have built AISHA as one large prompt around a spreadsheet and some pasted handbook text. That would have been faster at the start, but it would not have shown the real design problem. Onboarding support needs grounding, memory, action, escalation, and monitoring. It also needs boundaries. The architecture therefore separates documents from state, language generation from deterministic updates, support signals from private transcripts, and demo-time simulation from real-world claims.

In the end, the architecture is a map of the product thesis. AISHA helps Alyssa move toward Day 30 readiness by keeping the handbook, ramp plan, human support network, and safety layer in the same loop. It gives HR enough signal to offer help, not enough detail to police her. That is the reason the system is built this way, and it is also the reason each technical choice matters.

4. Experiments and Evaluation

After the architecture was working, our evaluation question became practical: can AISHA's support loop be trusted enough for a student capstone demo? We did not try to prove production readiness. Instead, we tested the contracts that matter for Alyssa's Day 30 story: grounded answers, safe task updates, memory, guardrails, pulse signals, privacy-conscious monitoring, and a reusable API.

Table 1. Experiment summary

Experiment	What we tested	What passed	What failed	What we learned
Guardrail model ablation	`llama3.2:1b` vs `qwen2.5:3b-instruct` on a 15-case topic, multilingual, benefits, off-topic, and injection battery	`qwen2.5:3b-instruct` reached about 14 to 15 out of 15 and handled multilingual support questions better	`llama3.2:1b` scored 8 out of 15 and over-blocked benefits jargon and non-English on-topic messages	The smallest model was not the safest model. A guardrail must protect legitimate support requests, not only reject bad ones
Retrieval and citations	Common handbook questions such as leave, SSS deductions, pre-start tasks, MFA, AML, and benefits	Expected sources appeared, and the citation contract `[source: filename.md]` was enforced after generation	This was a small sanity set, not a large retrieval benchmark	RAG worked for the demo scale, but production would need permission filters, freshness checks, and stronger retrieval evaluation
Safe task mutation	Completing tasks by numeric ID, fuzzy title, and ambiguous wording	Clear requests updated SQLite, numeric IDs resolved ambiguity, and close matches refused to mutate	Ambiguity scoring is still a heuristic	Agent action needs ordinary deterministic code around it. AISHA should act when clear and pause when unsure
Agent, API, and memory loop	Fake tool-calling models, FastAPI dependency overrides, persisted chat messages, and plan changes	Tool calls, source capture, escalations, API responses, and chat memory worked without Ollama	Fake LLM tests do not prove real `llama3.1:8b` answer quality	Offline tests are still valuable because they verify the plumbing around the model
Pulse and privacy signals	Simulated-date scheduling, pulse parsing, risk flags, HR dashboard summaries, and observability logs	Pulse timing followed the demo clock, support flags worked, and JSONL logs stored metadata without raw message text	Raw pulse replies are still stored locally for demo drill-down	AISHA can show support signals without making the default HR view a private transcript archive
App reliability and story consistency	Streamlit headless boot, seed data, rebrand regression, observability error logging	The app can boot without Ollama, logs tolerate errors, and stale pre-AISHA wording is guarded against	This does not replace live demo testing with the real local model	The product story is part of the system, so reliability includes both code wiring and narrative consistency

The main pattern is that most tests avoid calling Ollama. That was intentional, not a shortcut. Local models are slow, hardware-dependent, and sometimes inconsistent, so we tested parsing, state, API behavior, citation enforcement, pulse logic, and observability with fake LLMs where possible. The real model still matters, but the automated suite gives us a stable way to know whether the support loop around the model is intact.

The experiments also made our tradeoffs clearer. We learned that the guardrail model needed to be stronger than originally planned, that citation enforcement should be checked after generation, and that tool use needs deterministic safety checks before updating Alyssa's ramp plan. We also learned that observability can be useful without becoming surveillance, as long as it logs route, latency, tools, sources, and errors instead of raw conversation text.

Ultimately, these experiments do not prove AISHA would reduce time-to-ramp in a real BDO environment. The data is fictionalized, the handbook is small, and there are no real BDO systems,

HRIS records, SSO, LMS, or production access controls. What they do prove is narrower but meaningful: the prototype can connect knowledge, action, memory, guardrails, pulse signals, and human escalation in one local loop that supports Alyssa's path toward Day 30 readiness.

5. Retrospective

What worked best was starting from Alyssa instead of starting from the model. Once we knew she was a Management Trainee and Branch Banking Associate moving toward a Day 30 Readiness Check, the technical decisions became easier to judge. RAG mattered because she needed grounded answers. Tools mattered because she needed progress updates and human routing. Pulse mattered because a new hire can struggle quietly before anyone notices. The clearest win was that AISHA became more than a chatbot without becoming a huge enterprise system.

What surprised us was how much of the project was not really about making the model smarter. A lot of the trust came from plain engineering choices: citation enforcement, deterministic task matching, SQLite state, fake LLM tests, and logs that avoid raw message text. We expected the agent prompt to be the center of the work, and it was important, but the surrounding contracts did just as much to make AISHA feel responsible. In a way, the model became one part of the system rather than the whole magic trick.

What felt harder than expected was keeping the product humane while still making it measurable. It is easy to say "support signals, not surveillance," but harder to design every screen, log, pulse summary, and escalation around that promise. The pulse feature especially forced us to be careful. We wanted AISHA to notice when Alyssa might need help, but we did not want to turn her private uncertainty into a performance label. That tension made the project more real, because HR AI is not only a technical problem. It is also a trust problem.

With more time or budget, we would improve the parts that a real organization would care about first: consent, access control, retention rules, SSO, HRIS and LMS integrations, stronger retrieval evaluation, better UI design, and real user testing with new hires or HR mentors. We would also move from SQLite to a production database, add role-based permissions, and test whether AISHA actually reduces time-to-ramp instead of only demonstrating the loop. The next version should measure outcomes, not just capabilities.

What we would not overbuild again is infrastructure before the story is clear. It was tempting to imagine dashboards, integrations, analytics, and production deployment too early. In hindsight, the better move was to keep returning to one question: does this help Alyssa take the next useful step? If the answer was no, it could wait. That lesson may be the most useful part of the project. Agentic AI does not need to feel huge to be meaningful. It needs to be grounded, careful, and pointed at a real human moment.

References

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). *Retrieval-augmented generation for large language models: A survey*. arXiv. https://arxiv.org/abs/2312.10997

Ju, A., Sajnani, H., Kelly, S., & Herzig, K. (2021). *A case study of onboarding in software teams: Tasks and strategies*. arXiv. https://arxiv.org/abs/2103.05055

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459-9474. https://arxiv.org/abs/2005.11401

Maghsoudi, M., Kamrani Shahri, M., Agha Mohammad Ali Kermani, M., & Khanizad, R. (2023). *Mapping the landscape of AI-driven human resource management: A social network analysis of research collaboration*. arXiv. https://arxiv.org/abs/2308.09798

Mantello, P., Ho, M.-T., Nguyen, M.-H., & Vuong, Q.-H. (2021). *My boss the computer: A Bayesian analysis of socio-demographic and cross-cultural determinants of attitude toward the non-human resource management*. arXiv. https://arxiv.org/abs/2102.04213

National Institute of Standards and Technology. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). https://doi.org/10.6028/NIST.AI.100-1

Shneiderman, B. (2020). Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human-Computer Interaction, 36*(6), 495-504. https://doi.org/10.1080/10447318.2020.1741118

Whalley, J., Imbulpitiya, A., Clear, T., & Ogier, H. (2024). *From student to working professional: A graduate survey*. arXiv. https://arxiv.org/abs/2410.07560

Appendix A. Implementation Evidence Map

This appendix keeps the main write-up readable while preserving the technical evidence behind the implementation claims.

Claim Area	Evidence in Project	Why It Matters
RAG with citations	`ingestion.py`, `retriever.py`, Chroma, citation guardrail tests	Keeps onboarding answers grounded instead of relying on model memory.
Agent and tool use	`agent.py`, `tools.py`, fake tool-calling smoke tests	Shows AISHA can search, read plans, update tasks, route people, and escalate.
Persistent state	`state.py`, SQLite tables, memory tests	Lets Alyssa's progress and chat context survive beyond a single turn.
Guardrails	`guardrails.py`, parse and PII redaction tests	Keeps the assistant scoped to onboarding and safer with sensitive-looking output.
Pulse support signals	`pulse.py`, dashboard behavior, pulse tests	Supports the privacy thesis: HR sees signals and summaries rather than raw transcripts by default.
API and observability	`api.py`, `service.py`, `observability.py`, API and logging tests	Proves the guarded loop can be reused outside Streamlit and inspected without logging message text.